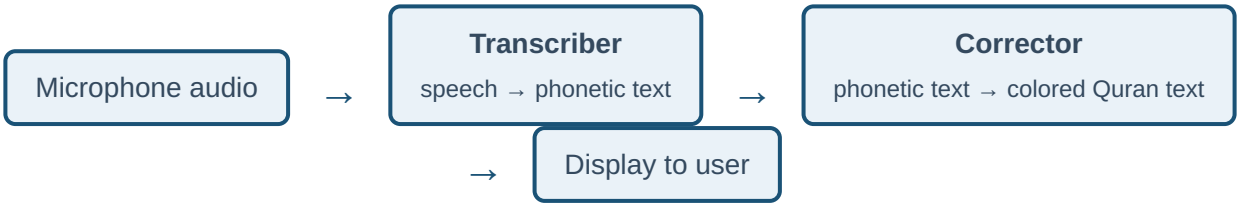
phonetizer — Developer Guide

1. The big picture

This module powers a **Quran recitation correction app**. The idea is simple: a person recites the Quran out loud, and the app tells them in real time which parts they recited correctly and which parts they got wrong.

Under the hood, two things happen in sequence:



1. The **Transcriber** listens to the microphone and converts the audio into a special phonetic text — a representation of what the person is saying, written in a phonetic alphabet designed for Quranic Arabic. You don't need to understand this format; just treat it as an opaque string that flows from one component to the next.
2. The **Corrector** takes that phonetic text, figures out which verse of the Quran the person is reciting, and compares what they said to the actual text. It returns the Quran text along with a color for each character: green if correct, orange if slightly off, red if missing.

Both objects are **stateful**. They remember everything from earlier in the session. You create them once when the user starts reciting, feed them data as it comes, and throw them away when the session ends.

2. Why JavaScript?

The original code was written in Python. Python is great for prototyping, but it's a poor fit for a production mobile app:

- **Size.** Bundling a Python interpreter and its libraries (NumPy, ONNX Runtime, kaldi-native-fbank) adds roughly **50 MB** to the app.
- **Startup time.** Launching Python and importing those libraries takes several seconds before the app can process the first audio frame.
- **Speed.** Python's interpreted loops are very slow compared to JavaScript running on a JIT-compiled engine like V8 or Hermes.

This folder is a complete JavaScript rewrite. It needs only one native library (`onnxruntime-node`, for running the neural network). Everything else — the audio signal processing, the text processing, the Quran search index — is implemented in pure JavaScript.

3. What's in the folder

You only need to know about two files:

File	What it does
`model_api.js`	Exports the `Transcriber` class (audio → phonetic text)
`correction.js`	Exports the `Finder` and `Corrector` classes (phonetic text → correction)

The folder also contains data files that these classes need at runtime:

File	Purpose
`comp_graph.onnx`	The neural network model
`meta_data.json`, `vocab.json`	Configuration files for the model
`quran.json`	The full Quran text (Warsh riwaya)

All other `.js` files in the folder are internal helpers. You never import them directly.

4. The Transcriber

The Transcriber converts audio into phonetic text. Think of it as a live subtitling engine: you feed it audio, and at any point you can ask "what has been said so far?"

Step 1 — Create and initialize

Point it to the three data files and call `init()`. This loads the neural network into memory. Do this once, before you start processing audio.

```
const { Transcriber } = require("./phonetizer/model_api");

const transcriber = new Transcriber(
  "./phonetizer/comp_graph.onnx",
  "./phonetizer/meta_data.json",
  "./phonetizer/vocab.json"
```

```
);
await transcriber.init();
```

Step 2 — Feed audio

As audio arrives from the microphone, pass it to `push()` immediately. The audio must be **mono, 16 kHz, float32** samples (values between −1 and 1). Don't buffer — push every chunk as soon as your audio layer delivers it, no matter how small.

```
// 'samples' comes from your audio capture layer (e.g. a WebAudio callback)
await transcriber.push(Array.from(samples));
```

Step 3 — Read the result

At any point, call `read()` to get the phonetic text accumulated so far. This is a cheap call — it doesn't do any computation, it just returns the current text.

```
const phoneticText = transcriber.read();
// phoneticText is a string like "sنِاهِررَحمَا بِسمِللَا..."
// You don't need to parse or understand it — just pass it to the Corrector.
```

Important: The Transcriber is stateful. Audio chunks must arrive in order, and you cannot rewind. When the user stops and starts a new recitation session, create a fresh `Transcriber` instance or call `reset()`.

5. The Corrector

The Corrector is the core of the app. It takes the phonetic text from the Transcriber and answers three questions:

1. **Where in the Quran** is the reciter right now?
2. **What is the actual Quran text** (in uthmani script) for that passage?
3. **Which characters did the reciter get right**, and which did they get wrong?

Step 1 — Create a Finder

Before you can correct anything, you need a `Finder`. The Finder builds a search index over the entire Quran so the Corrector can quickly locate any passage. This is a one-time cost:

- **First run:** takes about 3 seconds to build. The result is saved to a cache folder on disk.
- **Subsequent runs:** loads from cache in milliseconds.

```
const { Finder, Corrector } = require("./phonetizer/correction");

const finder = new Finder(
  "./phonetizer/quran.json",            // Quran text
  "./phonetizer/Data_for_quran_search"  // cache folder (created automatically)
);
```

You typically create one Finder when the app starts and reuse it for every session.

Step 2 — Create a Corrector

Create one Corrector per recitation session:

```
const corrector = new Corrector(finder);
```

If you know which verse the reciter will start from, you can tell the Corrector so it doesn't have to guess:

```
// The user chose to start from Sura 2, Aya 255 (Ayat al-Kursi):
const startCursor = finder.verseToCursor([2, 255]);
const corrector = new Corrector(finder, startCursor);
```

Step 3 — Call `correct()`

Every time you get new phonetic text from the Transcriber, pass it to `correct()`:

```
const [uthmaniText, colors, additions] = corrector.correct(phoneticText);
```

This returns three things:

Return value	What it is
`uthmaniText`	A `string` containing the Quran text (uthmani script) that corresponds to what the reciter is saying. This is what you display on screen.
`colors`	An array of numbers, **one per character** of `uthmaniText`. Each number tells you how to color that character: `1` = **correct** (green) `0` = **slightly off** (orange — e.g. wrong duration or slight mispronunciation) `-1` = **missing / skipped** (red)
`additions`	

	Positions where the reciter inserted words that are not in the Quran. Not used at the moment — you can ignore this array.

Example: if `uthmaniText` is `"بِسْمِ ٱللَّهِ"` and `colors` is `[1,1,1,1,1,-1,-1,-1,-1,-1]`, it means the reciter got the first five characters right (green) but skipped the rest (red).

Understanding the cursor

The Corrector needs to know *where* in the Quran the reciter is, so it can compare what they say to the right passage. It tracks this with a number called the **cursor**.

Think of the entire Quran written out as one long line of text, from the first verse of Al-Fatiha to the last verse of An-Nas. The cursor is simply a position in that line — like a bookmark that moves forward as the reciter progresses.

- The cursor **advances automatically** with each call to `correct()`.
- You can read it at any time: `corrector.cursor`
- To find out which sura and aya the cursor is in, call: `corrector.getAya(phoneticText)`, which returns `[[sura, aya], score]`.

You rarely need to manipulate the cursor yourself. The only time you set it manually is at construction, if you know the starting verse (as shown above).

> **Important:** Like the Transcriber, the Corrector is stateful. The same instance must be used for the entire recitation session. Each `correct()` call builds on the previous one. If you create a new Corrector mid-session, it loses track of where the reciter is and has to re-discover the position from scratch.

6. Full example

Here is a minimal working example that ties everything together:

```
const { Transcriber } = require("./phonetizer/model_api");
const { Finder, Corrector } = require("./phonetizer/correction");

async function startSession(startSura, startAya) {

  // === Setup (do this once when the app launches) ===

  // Build / load the Quran search index
  const finder = new Finder(
    "./phonetizer/quran.json",
    "./phonetizer/Data_for_quran_search"
```

```
  );

  // Load the speech recognition model
  const transcriber = new Transcriber(
    "./phonetizer/comp_graph.onnx",
    "./phonetizer/meta_data.json",
    "./phonetizer/vocab.json"
  );
  await transcriber.init();

  // Create a corrector starting at the chosen verse
  const cursor = finder.verseToCursor([startSura, startAya]);
  const corrector = new Corrector(finder, cursor);


  // === Audio loop (called repeatedly as audio comes in) ===

  async function onAudioChunk(samples) {
    // 1. Feed audio to the transcriber
    await transcriber.push(Array.from(samples));

    // 2. Get the phonetic text (what has been said so far)
    const phoneticText = transcriber.read();

    // 3. Compare to the Quran
    const [uthmaniText, colors, additions] = corrector.correct(phoneticText);

    // 4. Find which verse we're in
    const [[sura, aya], score] = corrector.getAya(phoneticText);

    // 5. Now render: display uthmaniText on screen,
    //    coloring each character i according to colors[i].
    //    (1 = green, 0 = orange, -1 = red)
  }

  return onAudioChunk;
}

// Usage: start reciting from Sura 1, Aya 1
const processChunk = await startSession(1, 1);

// Then, every time your audio layer delivers a new chunk:
// await processChunk(audioSamples);
```

7. Quick reference

	Transcriber	Corrector
Input		

	Audio samples (float32[], 16 kHz, mono)	Phonetic text string (from Transcriber)
Output	Phonetic text string	Quran text + colors + additions
Stateful?	Yes — one instance per recording session	Yes — one instance per recitation session
Async?	Yes (`init()`, `push()`)	No (synchronous)